Data Preparation and Supervised Baselines for Failure Prediction in CartPole Trajectories

Miguel Angel Torres Ramirez
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: migatorresr@udistrital.edu.co

Cristian David Romero Gil
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: cdromerog@udistrital.edu.co

Omar Esteban Peña Sanchez
Dept. of Computer Engineering
Universidad Distrital Francisco José de Caldas
Email: oepenas@udistrital.edu.co

***Abstract*—This report presents the data preparation pipeline and the supervised baselines for a binary failure-prediction task built on CartPole trajectories (35,874 samples from 400 episodes). The label marks the last five steps of every episode that ends in failure, which yields a 17.2:1 class imbalance. We describe the exploratory analysis, an episode-level split that prevents leakage between consecutive samples, a physics-informed feature set, and three traditional classifiers: Logistic Regression, Random Forest and Gradient Boosting. On the held-out test episodes, Gradient Boosting obtains the best results (F1 = 0.860, PR-AUC = 0.935), while Logistic Regression only becomes competitive (ROC-AUC 0.981 versus 0.542) after feature engineering. These results establish the reference performance for the deep learning models of the next workshops.**

***Index Terms*—CartPole, failure prediction, class imbalance, exploratory data analysis, Random Forest, Gradient Boosting, baselines.**

I. Introduction

Anticipating that a control system is about to fail is a natural supervised learning problem: given the current state of the system, a classifier must raise an alarm a few steps before the failure occurs. In this workshop we study this problem on trajectories of the CartPole system [1], where a pole must be kept upright on a moving cart.

The central question is: *what does the data look like, and what performance can well-engineered traditional models achieve?* The answer fixes the difficulty of the problem, the degree of class imbalance and the reference metrics against which the deep learning models of later workshops will be compared.

The contributions of this report are: (i) a verified and characterised dataset; (ii) a leakage-free preprocessing pipeline based on an episode-level split; (iii) three baselines evaluated with metrics suited to imbalanced data; and (iv) two controlled ablations that isolate the effect of feature engineering and of data augmentation.

The rest of the paper is organised as follows: Section II describes the dataset and its exploratory analysis; Section III presents the preprocessing pipeline; Section IV details the baseline models and the evaluation protocol; Section V reports and discusses the results; and Section VI concludes.

TABLE I
DATASET SUMMARY

Property	**Value**
File size	1.99 MB
Rows / episodes	35,874 / 400
Missing values / duplicates	0 / 0
Class 0 (safe)	33,904 (94.51%)
Class 1 (failure imminent)	1,970 (5.49%)
Imbalance ratio	17.2 : 1
Episode length (min / median / max)	8 / 34.5 / 500

II. Dataset and Exploratory Data Analysis

A. Dataset Description

The dataset contains one row per time step of each simulated episode. Each row stores the episode identifier, the time index `t`, the cart position `x`, cart velocity `x_dot`, pole angle `theta`, pole angular velocity `theta_dot`, the `action` taken (0/1) and the binary target `label`. The task is binary classification. The dataset was generated by rolling out the CartPole-v1 environment (OpenAI Gym 0.26.2) with a mixed policy (noisy expert plus random actions), with all seeds (Python, NumPy, Gym) fixed to 42. Each state is labelled 1 if the episode physically fails within the next five steps ($|\theta|$ beyond 12° or $|x| > 2.4$ m). The environment is free and pip-installable, with no license restrictions for this use.

Integrity was verified with the SHA256 checksum (`ec067d5d...f53f8`), which matches the expected value. Table I summarises the dataset, as printed by `dataset_info.py`.

B. Target Definition and Class Imbalance

The label equals 1 exactly in the last five steps of the 394 episodes that end in failure. The remaining six episodes reach the 500-step limit and contain no positive sample. With a 17.2:1 ratio, a trivial classifier that always predicts "safe" already reaches 94.5% accuracy, so accuracy cannot be used as a decision metric.

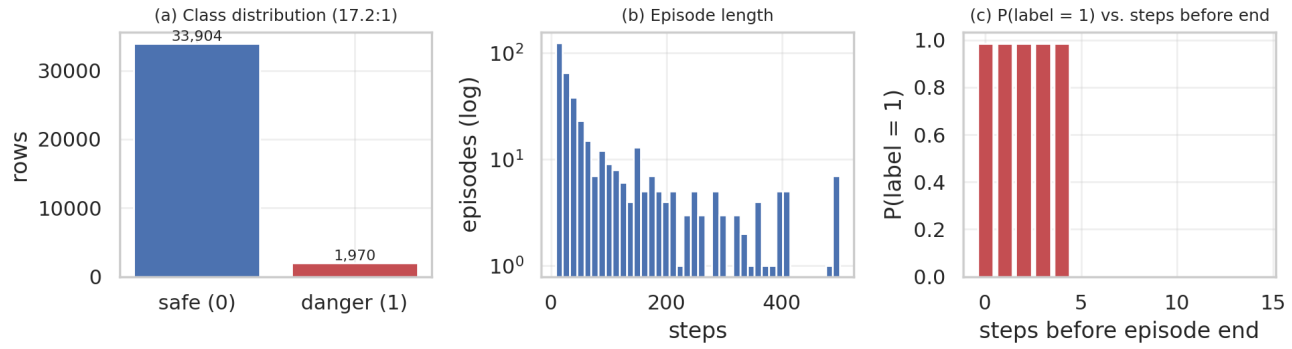


Fig. 1. Target distribution, episode-length distribution (log scale) and label probability as a function of the steps remaining before the end of the episode.

TABLE II
EPISODE-LEVEL SPLIT (SEED 42)

Partition	Episodes	Rows	Positives
Train	279	24,782	5.57%
Validation	61	5,762	5.21%
Test	60	5,330	5.44%

C. Feature Statistics and Data Quality

The four state variables are well bounded: `x` $\in [-2.40, 2.40]$, `theta` $\in [-0.209, 0.209]$ rad, and the velocities stay within roughly ± 3. No missing values, duplicates or corrupted rows were found, so 100% of the samples are usable. The IQR rule flags 26.4% of the `x` values as outliers, but this reflects the wide spread of the cart position, not erroneous data; the other state variables show between 3.0% and 4.3% flagged values. All stored states lie inside the standard failure limits ($|\theta| < 0.2095$ rad, $|x| < 2.4$ m); the terminal state is not recorded.

D. Feature Relationships

The Pearson correlation between each raw feature and the label is very weak (between -0.036 and 0.015), which indicates that no raw variable separates the classes linearly. Fig. 2 shows the class-conditional distributions and Fig. 3 the correlation matrix.

III. Preprocessing Pipeline

A. Episode-Level Split

Consecutive rows of an episode are nearly identical, so a random split by row would place almost duplicated states in train and test and inflate the metrics. We therefore split the 400 *episodes* into 70/15/15%, stratified by episode-length quartile because the lengths are strongly skewed (Table II). The split is stored in `data/splits.json` and is never recomputed.

B. Feature Engineering

Because an episode fails when $|\theta| > 0.2095$ rad or $|x| > 2.4$ m, the features describe the distance to those limits and the speed of approach. All of them use only the current and past steps of the same episode (no information from the future), and the time index `t` is excluded. The final 16 features are:

- **Raw (5):** `x`, `x_dot`, `theta`, `theta_dot`, `action`.
- **Distance to limits (4):** $|x|$, $|\theta|$, x-margin and θ-margin.
- **Short-term dynamics (3):** first differences of x, θ and $\dot{\theta}$.
- **Rolling windows (2):** 3-step mean of $|\theta|$ and $|\dot{\theta}|$.
- **Time to limit (2):** seconds to reach each limit assuming constant velocity.

The engineered features correlate far more with the label than the raw ones (e.g., 0.561 for the rolling mean of $|\dot{\theta}|$ and 0.472 for $|\theta|$).

C. Normalisation, Missing Data and Augmentation

Features are standardised with z-scores because their ranges differ widely and the linear model is scale-sensitive; the mean and standard deviation are computed *only* on the training set and then applied to validation and test. There are no missing values, so no imputation is needed. As an optional augmentation, applied to the training set only, we create one noisy copy of each training episode by adding Gaussian noise equal to 2% of the standard deviation of each state variable, which emulates sensor imprecision while keeping the label.

IV. Baseline Models and Evaluation Protocol

A. Models

Three traditional classifiers were trained on the scaled features (scikit-learn [2]):

- **Baseline 1 – Logistic Regression:** linear model with balanced class weights.
- **Baseline 2 – Random Forest** [3]: 300 trees, balanced-subsample class weights.
- **Baseline 3 – Gradient Boosting** [4]: histogram-based implementation, 300 iterations, learning rate 0.1, balanced sample weights.

B. Evaluation Protocol

Models are fitted on the training set. The decision threshold is selected on the validation set by maximising F1, so the test set is never used for tuning. We report accuracy, precision, recall, F1, ROC-AUC, PR-AUC and the confusion matrix. For this task a missed failure (false negative) is the costly error, so **recall** is the key metric, balanced by precision to limit false

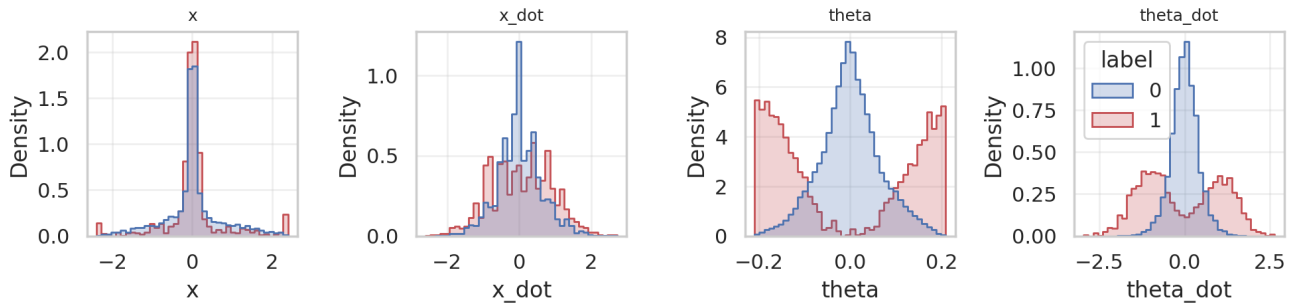


Fig. 2. Distribution of the state variables by class (density, label 0 = safe, 1 = danger).

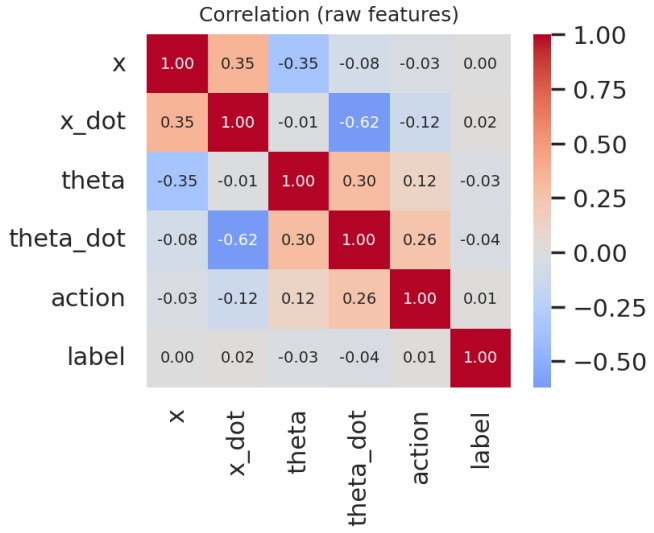


Fig. 3. Correlation matrix of the raw features and the label.

TABLE III
BASELINE RESULTS ON THE TEST SET (ENGINEERED FEATURES)

Model	Prec.	Rec.	F1	ROC-AUC	PR-AUC
Logistic Regression	0.669	0.703	0.686	0.981	0.791
Random Forest	0.833	0.841	0.837	0.994	0.923
Gradient Boosting	0.834	**0.886**	**0.860**	**0.996**	**0.935**

alarms; PR-AUC is preferred over ROC-AUC for summarising performance under a 17:1 imbalance [5]. Accuracy is reported only for completeness.

V. Results and Discussion

A. Baseline Comparison

Table III reports the test-set results (60 episodes, 5,330 rows, 290 positives) at the validation threshold. Fig. 4 shows the confusion matrices.

Gradient Boosting detects 257 of the 290 positive samples with 51 false alarms (TN = 4,989, FP = 51, FN = 33, TP = 257), followed by Random Forest (244 detected, 49 false alarms) and Logistic Regression (204 detected, 101 false alarms). The accuracy of all models (0.965 to 0.984) is above the 0.945 of the trivial classifier but hides the differences in the minority class, which confirms that accuracy is not informative here.

The decision threshold matters. At the default 0.5, Logistic Regression reaches a recall of 0.962 but a precision of only 0.361 (F1 = 0.525), i.e., many false alarms; the threshold selected on validation (0.932) trades part of that recall for a much better F1 (0.686).

TABLE IV
ABLATION: F1 / PR-AUC ON THE TEST SET

Model	Raw feat.	Engineered	Eng. + aug.
Logistic Regression	0.230 / 0.098	0.686 / 0.791	0.685 / 0.792
Random Forest	0.839 / 0.944	0.837 / 0.923	0.826 / 0.933
Gradient Boosting	0.873 / 0.952	0.860 / 0.935	0.848 / 0.942

B. Ablation: Feature Engineering and Augmentation

Table IV changes one factor at a time with respect to the main setting (engineered features, no augmentation).

Feature engineering. For Logistic Regression the engineered features raise the ROC-AUC from 0.542 (almost random) to 0.981 and the PR-AUC from 0.098 to 0.791: the raw decision boundary is not linear, but distances to the failure limits make it nearly linear. For the tree ensembles the raw features are already enough, and the differences between raw and engineered inputs (up to 0.013 in F1) are small relative to what a single split of 60 test episodes can resolve, so we do not claim that either is better for these models.

Augmentation. Noise augmentation did not improve any model (F1 differences between -0.012 and -0.001). With a clean simulator, the added noise does not seem to provide useful variety.

C. Problem Difficulty

Tree ensembles reach a ROC-AUC close to 0.995, so the classes are almost separable in a ranking sense, yet the F1 stays around 0.84–0.87: the remaining errors concentrate on the precision/recall trade-off of the rare class. A plausible reason is that states a few steps before the label window are almost identical to states inside it (the label boundary is arbitrary with respect to the dynamics), but we did not verify this and leave it as a hypothesis. The strong results also leave limited

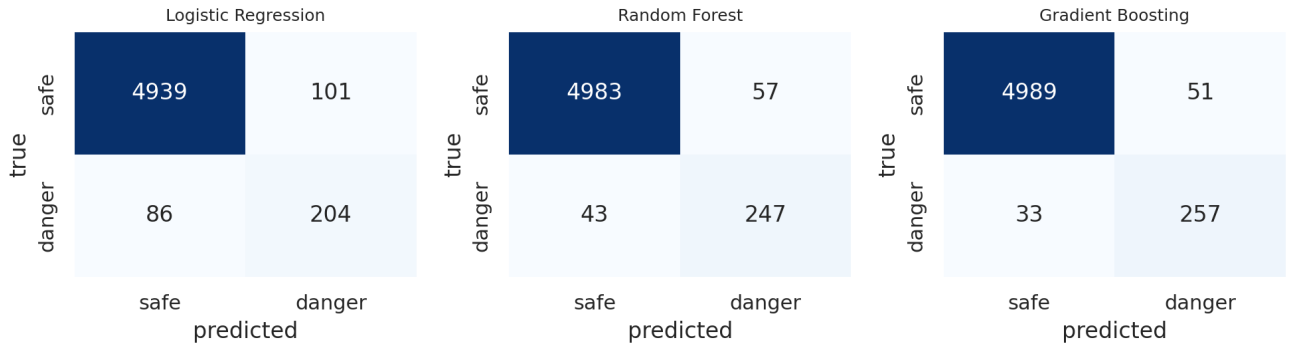


Fig. 4. Confusion matrices of the three baselines on the test set (threshold selected on validation).

headroom for the deep learning models of the next workshop, which will have to be compared against a high baseline.

D. Limitations

All results come from one episode-level split with a single seed and 60 test episodes, so small differences (e.g., Random Forest versus Gradient Boosting) are not conclusive. Cross-validation grouped by episode and several seeds would give variance estimates. Hyperparameters were not tuned beyond sensible defaults, and the dataset covers only the behaviour of the mixed policy (noisy expert plus random actions) that generated it.

VI. Conclusions

We characterised a CartPole failure-prediction dataset, identified a 17.2:1 imbalance and the need to split by episode, and built a reproducible pipeline with scaling and augmentation restricted to the training data. Gradient Boosting was the strongest baseline (F1 = 0.860, recall = 0.886, PR-AUC = 0.935), Random Forest was close, and Logistic Regression required engineered distance-to-limit features to be useful. Noise augmentation brought no gain. These numbers are the reference for the next workshops. Future work includes grouped cross-validation, a multi-seed evaluation, and sequence models that exploit the temporal structure of the episodes.

Reproducibility

Code, split file and instructions are available at https://github.com/Tonyaa11/Cartpole.git. All reported values are produced by `src/baselines.py` (seed 42).

References

[1] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-13, no. 5, pp. 834–846, 1983.

[2] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[3] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[4] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Ann. Statist.*, vol. 29, no. 5, pp. 1189–1232, 2001.

[5] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.